

Profiling and quantization

Measure where the time and memory go, fix the real bottleneck, then store each weight in fewer bytes.

THE QUESTION

The model works, but the API is slow and the file is too big. What do you change first?

THE INTUITION

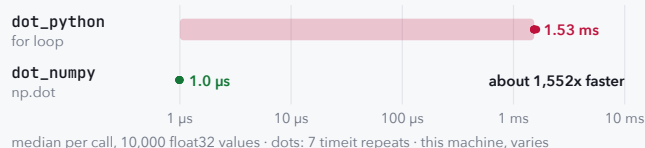
A profiler is an itemized receipt: the time often goes to reloading files, not to predict. A model is a bag of numbers, so its size is parameters × bytes; INT8 keeps 1 byte per weight plus a shared scale.

KEY TAKEAWAY

Profile before optimizing. Load once and batch, then quantize, and re-measure size, latency and accuracy after each change.

%timeit → cProfile → line_profiler → load once · batch → quantize → measure again

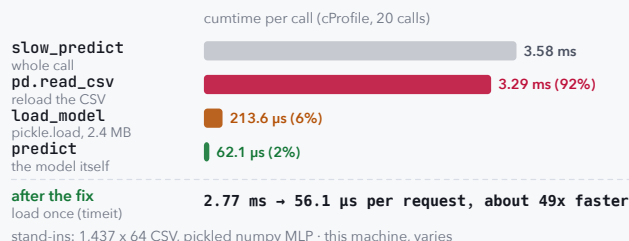
1 Time many runs, not one



```
t0 = time.perf_counter()
model.predict(X_test) # one run: noisy
print(time.perf_counter() - t0)
%timeit model.predict(X_test) # Jupyter
```

%timeit times a cell; scripts use `timeit.timeit(f, number=100)`. Warm up first; on a GPU, call `torch.cuda.synchronize()` before reading the clock.

2 cProfile shows which function eats the time



```
cProfile.run("slow_predict(sample)",
              sort="cumulative")
```

`tottime`: inside the function only; `cumtime`: plus everything it calls. For a loop of calls, wrap it in `prof.enable()` and `prof.disable()`, then sort with `pstats`.

3 Load the model once, at startup

```
model = joblib.load("model.pkl") # once

@app.post("/predict")
def predict(msg):
    # not here: a load on every request
    return model.predict([msg.text])
```

The lecture's receipt with loads in the handler: `read_csv` 1.8 s + `joblib.load` 0.6 s + `predict` 0.001 s = 2.4 s a request; after the move, about 1 ms. In Streamlit, `@st.cache_resource` (web apps sheet).

4 Line and memory profilers go one level deeper

```
df = pd.read_csv(f)
float64 DataFrame
X = df.values
a view here: no copy
X_s = (X - mu) / sd
new array; peak 24 MB
read_csv(f, dtype="float32")
half the bytes
```

traced memory after the step

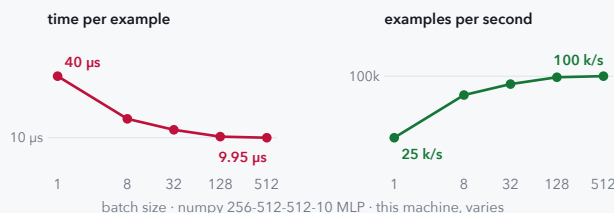
8.0 MB
+0.0 MB
16.0 MB
4.0 MB

tracemalloc · 20,000 × 50 CSV · a mixed-dtype frame's .values does copy

```
pip install line_profiler memory_profiler
kernprof -l -v script.py # time per line
python -m memory_profiler script.py
```

Both read an `@profile` decorator. Each new array costs its full size; `tracemalloc` (standard library) gives current and peak bytes. Profilers slow code down, so switch them off after.

5 Batching pays the per-call cost once



```
for bs in [1, 8, 32, 128, 512]:
    x = torch.randn(bs, 256)
    t0 = time.perf_counter()
    for _ in range(200): model(x)
    dt = time.perf_counter() - t0
    print(bs, dt / 200 / bs) # s/example
```

Each call pays Python overhead, data transfer and a kernel launch. Per-example time falls until the arithmetic is the limit or memory runs out.

6 Fewer bits give fewer distinct values

FP32 4 bytes: 0 1 0 0 0 0 0 0 1 1 0 1 0
6.5 = 1.101 (binary) × 2⁴²: sign 0 · exponent 2 + 127 = 129 · mantissa 101
(the leading 1, is implicit) · about 4.3 billion values

FP16 2 bytes: 0 1 0 0 0 1 1 0 1 0 0 0 0 0 0 0
6.5 again: bias 15
65,536 values

INT8 1 byte: 0 0 1 0 1 0 1 0
42; -42 is 11010110 (two's complement)
256 values, -128 to 127

0.1 + 0.2 = 0.30000000000000004: compare floats with `np.isclose`, not ==

Training needs FP32, or updates near 10⁻⁷ round away. Inference is fine with FP16 on GPUs and INT8 on CPUs, phones and edge devices.

Store each weight in one byte

Size is parameters × bytes. INT8 keeps one float scale per tensor (or per row) and rounds every weight to its grid.

7 Size = parameters × bytes per parameter

bytes/param		64-128-10 MLP 9,610 params	BERT-base 110M params	LLaMA-7B 7B params	GPT-3 175B params
FP32	4	37.5 KB	440 MB	28 GB	700 GB
FP16	2	18.8 KB	220 MB	14 GB	350 GB
INT8	1	9.4 KB	110 MB	7 GB	175 GB
INT4	0.5	4.7 KB	55 MB	3.5 GB	88 GB

size = params × bytes per param · green: fits in 16 GB of laptop RAM · KB = 1024 B, GB = 10⁹ B

```
n = sum(p.numel()
        for p in model.parameters())
```

`nn.Linear(a, b)` holds $ab + b$ numbers: 64→128→10 has 8,320 + 1,290 = 9,610. llama.cpp's 4-bit LLaMA-7B (2023) fits a laptop. Q4_K_M.gguf marks a 4-bit GGUF file (llama.cpp, Ollama); -GPTQ and -AWQ name algorithms that pick the scales.

8 Symmetric INT8: one scale, then round



```
w = np.array([-0.81, -0.42, -0.05,
              0.12, 0.47, 0.79],
              dtype=np.float32)
scale = np.max(np.abs(w)) / 127
q = np.round(w / scale).astype(np.int8)
w_back = q.astype(np.float32) * scale
```

Stored: int8 values plus one float32 scale, 1 byte per weight instead of 4. Clip to $[-127, 127]$ when the scale comes from other data, as with activations. The matmul still runs in float, or in INT8 kernels that rescale.

9 Rounding error is at most half a step



One large outlier widens the step for the whole tensor. INT4 keeps 15 levels, so its scale is $\max|w|/7$; clamping to $[-7, 7]$ with the INT8 scale crushes large weights (max error 1.1 here).

10 A zero point and per-row scales use the grid better

ReLU outputs (all ≥ 0)

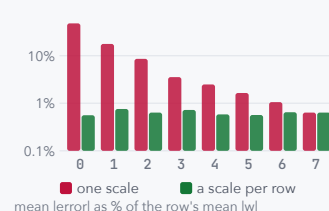
symmetric
93 levels used
0.0051

asymmetric
169 levels, zero point 0
0.0027

bar: mean |error|

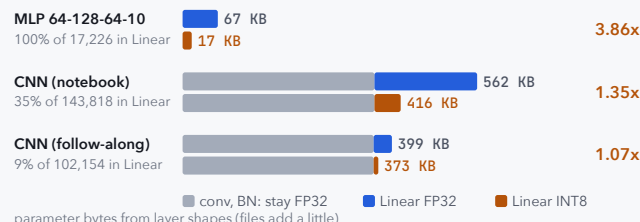
seeded data

8 × 64 matrix, rows 0.02 to 1.2



Asymmetric: $s = (x_{\max} - x_{\min})/255$, $z = \text{round}(-x_{\min}/s)$, $q = \text{round}(x/s) + z$ in 0 to 255. Symmetric scaling wastes the negative half on values ≥ 0 . Per-channel: one scale per output row, so small rows get their own fine step.

11 quantize_dynamic: one line, Linear layers only



```
from torch.ao.quantization import \
    quantize_dynamic # or torch.quantization
qmodel = quantize_dynamic(model.eval(),
                          {nn.Linear}, dtype=torch.qint8)
```

Weights turn INT8 once; each call measures the activations' range, quantizes them, multiplies in INT8 and rescales. Conv layers stay FP32, so a conv-heavy CNN barely shrinks. A tiny MLP in the lecture went 70.0 → 22.0 KB but 0.45 → 2.14 ms. Static quantization (calibration data) and quantization-aware training (retraining) cost more work.

12 Pause and think

INT8 0.958 vs FP32 0.955? One of 360 test digits is 0.28%: the same accuracy.

Quantized, and slower? Small matmuls, or a CPU without INT8 instructions. Memory-bound LLMs do speed up.

Phone or Java service? `torch.onnx.export`, then ONNX Runtime: no PyTorch needed.

Pruned 40%, same file size? Unstructured pruning leaves zeros in a dense matrix; structured pruning drops whole rows.

Much smaller still? Distil: train a small student on the teacher's softened probabilities.

measure first · load once · batch · size = params × bytes · $q = \text{round}(w / \text{scale})$ · re-check accuracy

Source: lectures "Profiling, quantization and model optimization" and "Model profiling and quantization", the profiling and quantization lab, its companion notebooks and follow-along, nipunbatra.github.io/stt-ai-teaching. Timings and memory were measured on one laptop with numpy, pandas, cProfile and tracemalloc (one BLAS thread) and vary; the endpoint uses stand-in files (a pickled numpy MLP for the random forest). The 2.4 s receipt, 70.0 → 22.0 KB and 0.45 → 2.14 ms are quoted from the lecture. Next: the building AI agents sheet.